AWS Resilience Hub — Data model for an operator UI

Official API analysis and logical entity relationship model — Reviewed 5 November 2026

Review status: reviewed against the detailed v2 operation contracts and corrected before the UI prototype. This revision repairs observation identity, makes service and snapshot foreign-key scopes explicit, separates summary collection from detail hydration, and defines aggregate evidence rules. See review.md for the decision log. The five diagrams are supplied as editable Mermaid and portable SVG; reviewed-report.pdf is the rendered report.

Recommended design: use the next-generation Resilience Hub API as the source for architecture, assessment, findings and dependency information. Add a temporal application layer for history and event processing, and use monitoring evidence for live operational health.

This report maps the ten requested entity concepts to current AWS response and mutation shapes, then proposes normalized relationships and the additional observations needed for an event-driven UI. AWS facts are cited by source IDs. Items described as **local**, **derived** or **proposed** are product design decisions, not additional AWS fields.

Requested concept	Current contract and modeling decision
SystemEntity	System / SystemSummary; retain systemId and ARN, including limited-visibility summaries.
UserJourneyEntity	UserJourney; parent supplied by request scope and service membership represented separately.
ServiceEntity	Service / ServiceSummary; primary assessment unit with multiple system and journey associations.
ResiliencePolicyEntity	Policy; four typed components and per-service effective target values.
AssessmentEntity	AssessmentSummary; service-scoped asynchronous run with status, cost and result counts.
FindingEntity	Finding / FindingSummary; service-scoped mutable finding, without an exposed assessment foreign key.
RecommendationEntity	Local normalization of three nested suggested-change groups; enrichment fields remain product-owned.
DependencyDiscoveryConfig	Embedded service value object; observed state and caller-settable configuration differ.
DependencyEntity	DependencySummary; DNS-oriented discovered dependency with criticality and usage data.
TopologyEntity	Local snapshot assembled from ListResources and ListServiceTopologyEdges.

The guide uses the requested Entity names, while current generated contracts use the formal shapes above. The official SDK identifies service resiliencehubv2, API version 2026-02-17. This analysis does not mix in the older application-centric API. [S01] [S02]

2. Contract boundaries and identity

Use detailed operation schemas as the integration contract

AWS overview pages and detailed schemas are not completely aligned. Performance targets, recommendation cost and complexity, dependency allowed status, and a versioned topology object appear in conceptual descriptions but are absent from the current shapes examined. Keep these as explicit extensions if the product needs them. Current operation names also differ from some overview examples. [S01] [S02] [S08] [S17] [S22] [S26]

Entity	Recommended identity	Identity rule
System	Local systemKey; retain systemId and nullable systemArn	The full System has an ARN. SystemSummary can omit it. Reconcile identifiers only within verified source scope; do not invent hidden account data.
User journey	(systemKey, userJourneyId)	Operations address a journey within a system; retain parent context even when absent from the body.
Service / policy	serviceArn / policyArn	Treat full ARNs as opaque provider identities. Names are display values, not joins.
Assessment / finding / dependency	(serviceArn, provider object ID)	Matches service-scoped addressing and prevents unsafe assumptions about global uniqueness.
Input source / service function	(serviceArn, inputSourceId or serviceFunctionId)	Separate child records preserve source context and support targeted loading.
Topology / recommendation	Local ID scoped to capture or observation	Neither native topology IDs nor native recommendation IDs are exposed in the examined shapes.

The System/SystemSummary distinction is material for organization views: systemId is exposed for cross-account use, whereas organization and OU metadata can depend on access. Keep partially visible references rather than deleting or fabricating their attributes. [S03] [S04]

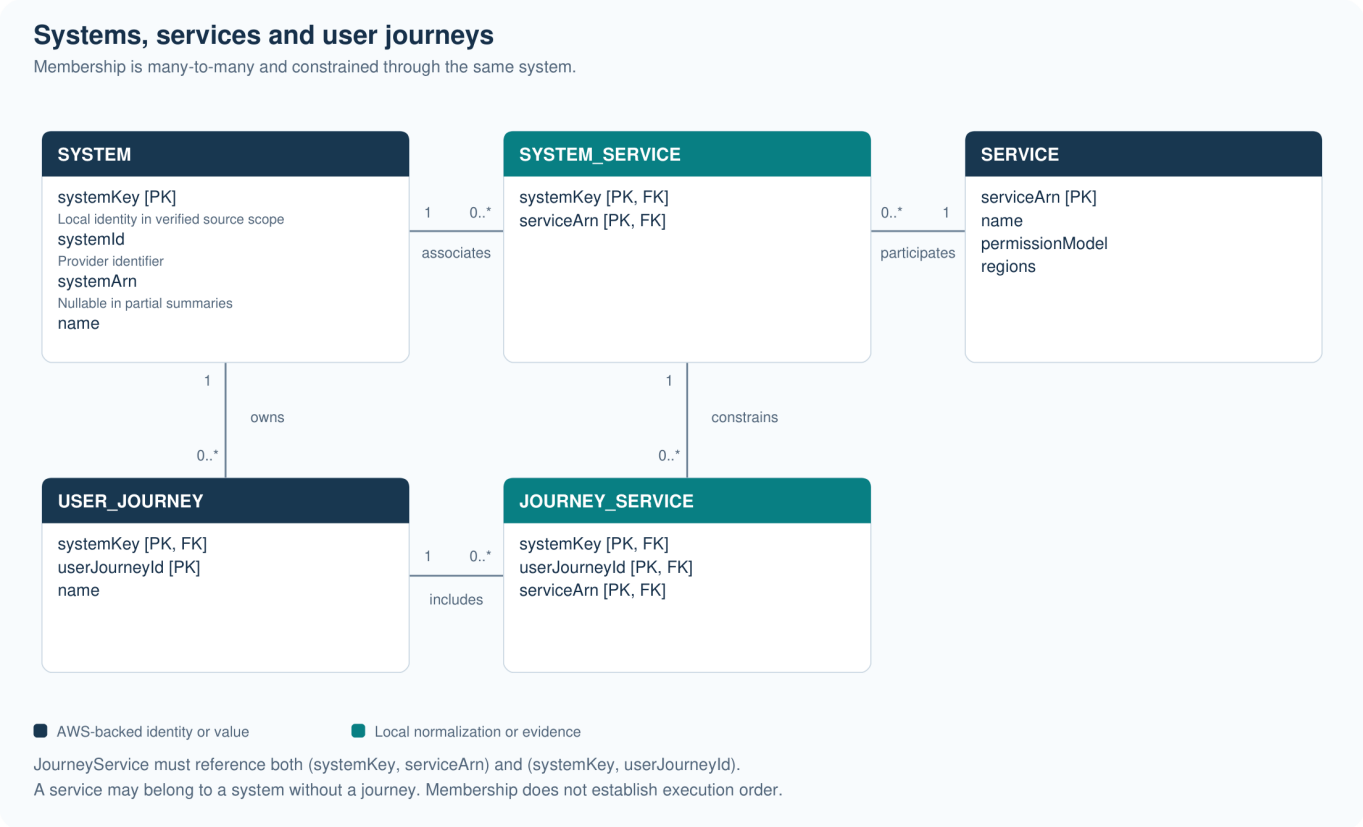
Common observation metadata

For every ingested object retain source account and Region or connection scope, API family/version, retrieval operation, observedAt, providerUpdatedAt when present, raw payload or payload reference, field presence, and a local projection revision. A timestamp that the API does not return must remain unknown. Normalize timestamps to UTC internally and localize display.

Preserve separate states for an omitted property, an explicit null, an empty collection, zero, disabled, inaccessible and stale. A lightweight list response must not erase fields previously obtained from a detailed Get. Incoming unknown enum strings should remain visible and diagnosable rather than crashing the projection.

Response optionality is not create-form optionality. For example, the Service response requires only its ARN and name, while CreateService requires name, permissionModel and regions. Generated command input types should drive editing forms; response schemas should drive read-model tolerance. [S06] [S07]

3. Systems services and user journeys



Editable Mermaid source · SVG

Figure 1. Normalized relationships derived from Service.associatedSystems. Endpoint labels show cardinalities. Join records are local relational representations of API arrays; they are not standalone AWS resources. [S05] [S06]

Entity	Principal fields beyond identity
SystemEntity	name; description; createdAt; updatedAt; organizationId; ouId; sharingEnabled; kmsKeyId; tags. Summary counts are projections.
UserJourneyEntity	name; description; policyArn; createdAt; updatedAt. The body has no embedded service list or parent system ARN. [S47]
ServiceEntity	name; description; accountId; organizationId; ouId; createdAt; updatedAt; regions; permissionModel; policyArn; reportConfiguration; tags; kmsKeyId; optional DevOps Agent integration.

A system has zero or more journeys. A service can participate in zero or more systems and zero or more journeys within each associated system. A system association with an empty journey list is valid. The source currently limits associatedSystems to 20 entries, and each association to 20 journey IDs. [S05] [S06]

Enforce JourneyService(systemKey, serviceArn) against SystemService, and JourneyService(systemKey, userJourneyId) against UserJourney. Stage unresolved references during partial or out-of-order ingestion; do not discard valid observations just because the related detail has not been fetched. Membership carries no documented journey execution order, branching, redundancy or critical-path weighting.

4. Policy structure and inheritance

Policy component	Typed values	Interpretation
availabilitySlo	target: number	Availability percentage; preserve unconfigured separately from zero.
dataRecovery	timeBetweenBackupsInMinutes: integer	Backup interval objective in minutes.
multiAz	rtoInMinutes; rpoInMinutes; disasterRecoveryApproach	Targets for recovery across availability zones.
multiRegion	rtoInMinutes; rpoInMinutes; disasterRecoveryApproach	Targets for recovery across Regions.

Policy additionally has policyArn, name, description, createdAt, updatedAt, tags, kmsKeyId, organizationId, sharingEnabled and associatedServiceCount. The last field is a returned aggregate. There is no current latency/error-rate PERFORMANCE component in this wire shape. [S08]

Model PolicyComponent with key (policyArn, componentType), or use four typed value objects. Avoid one unrestricted JSON blob if the UI must compare individual targets. Retain the original payload for compatibility. DR approach values are ACTIVE_ACTIVE, HOT_STANDBY, WARM_STANDBY, PILOT_LIGHT and BACKUP_AND_RESTORE. [S10]

Assignment and effective targets

A service and a journey each have zero or one directly assigned policy. A service can consequently have several applicable policies: its direct policy plus the policies of its journeys. AWS resolves conflicting requirements using the stricter value. The documented example chooses an RTO of 15 minutes over 30 minutes. There is no direct system policy field in the current System shape. [S03] [S08] [S11]

Use GetService.effectivePolicyValues as the authoritative returned effective target projection. Normalize its eight optional metrics: availabilitySlo; dataRecoveryTimeBetweenBackups; multiAzDrApproach; multiAzRpo; multiAzRto; multiRegionDrApproach; multiRegionRpo; multiRegionRto. Each returned value can have policyName and source. [S12]

Provenance trap: source is SELF or CROSS_ACCOUNT, describing account ownership. It does not mean direct versus inherited. The returned policyName is not a policy ARN. Any resolvedPolicyArn must be optional and based on a verified identity match. [S13]

Historical context

Capture effective target values and units, source policy name and ownership source, contributing policy references when known, membership context, capture time and hashes of the policy payloads observed. Policy content is mutable. A locally captured policy revision is not a provider revision identifier, and observing targets before a run does not prove they were the exact targets used by that run.

A service failing a stricter combined target does not establish that every looser journey policy failed. Show which service and effective requirement need attention; avoid claiming independently measured journey compliance from the service summary alone.

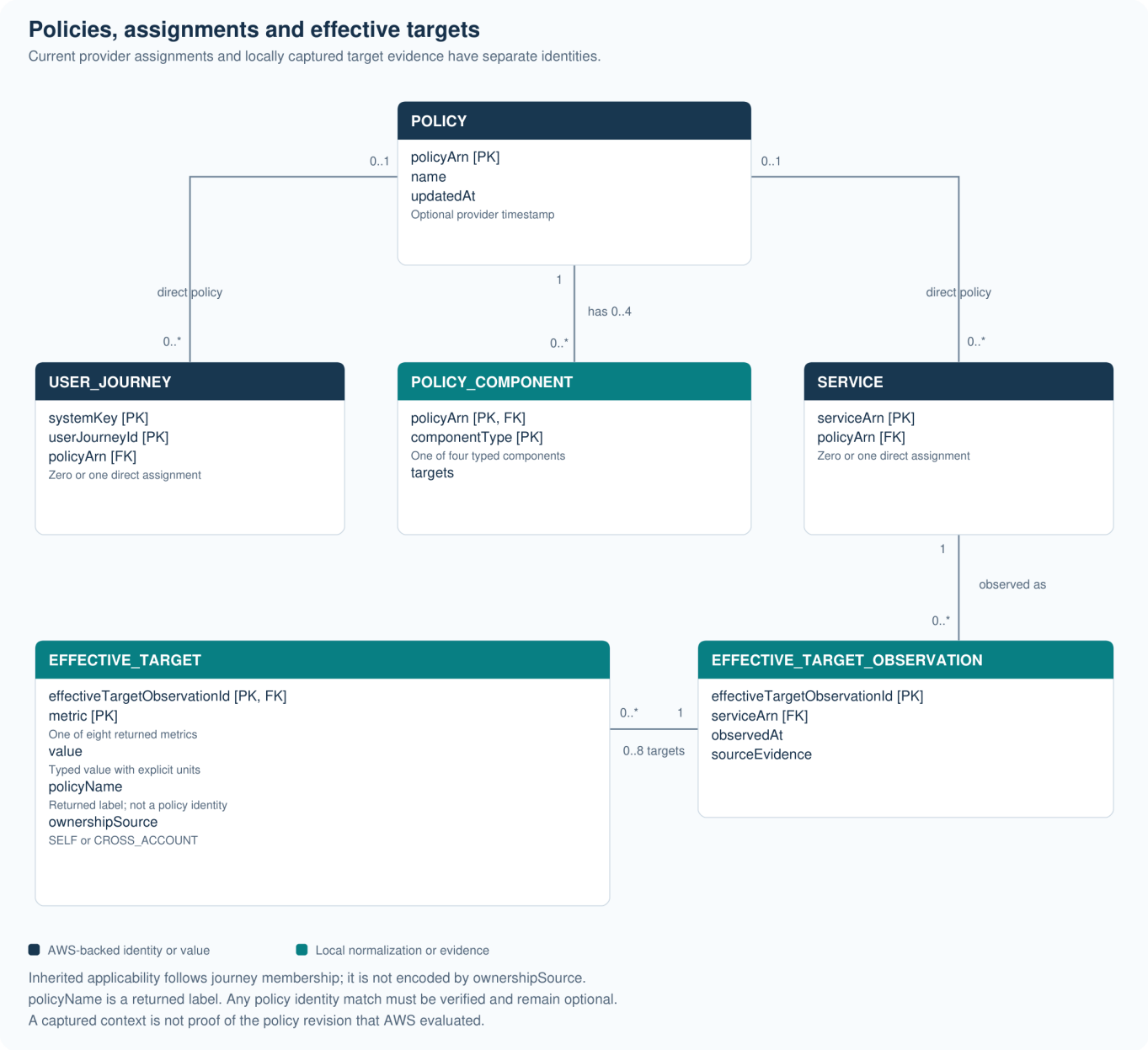
Reviewed policy observation model

`EffectiveTargetObservation(effectiveTargetObservationId, serviceArn, observedAt, sourceEvidence)` owns zero to eight `EffectiveTarget` rows keyed by `(effectiveTargetObservationId, metric)`. A missing metric remains absent; it is not a zero value. `PolicyComponent` is unique by `(policyArn, componentType)` and has at most four rows: `availabilitySlo`, `dataRecovery`, `multiAz`, and `multiRegion`. These are normalized forms of the returned value objects, not new AWS resource types.

If the application retains a `PolicyContextCapture`, give it its own local ID and record membership observations, observed policy content hashes, effective-target observation IDs, capture intervals and visibility. Keep `resolvedPolicyArn` nullable, together with the evidence used to resolve it. A matching name alone is insufficient, especially with cross-account policies. Store a separately evidenced direct/inherited assignment classification when needed; do not derive it from SELF or `CROSS_ACCOUNT`. [SO8] [S11] [S12] [S13]

Any assessment-to-context association is a **local temporal correlation**. Neither the observation ID nor a payload hash is an AWS policy revision or proof of the exact context evaluated in that assessment.

5. Policy entity relationship diagram



Editable Mermaid source · SVG

Figure 2. Native policy assignment is separate from locally retained EffectiveTargetObservation records. Each consumer has at most one direct policy; inherited applicability follows journey membership. Effective target policy names are retained without an unverified foreign key. [S08] [S11] [S12] [S13]

Supporting service configuration

PermissionModel is an execution/access configuration: invokerRoleName and optional crossAccountRoles, whose entries hold crossAccountRoleArn and optional externalId. It is not the new UI's authorization model. Store it as a subordinate value object and apply the application's normal access controls. [S14]

InputSourceEntity is service-scoped and separately listed. Retain inputSourceId, type, createdAt and the appropriate configuration: CloudFormation stack ARN, resource tags, EKS configuration, Terraform state file URL or design-file S3 URL. The read enum also contains MONITORING. Do not assume every read discriminator has a corresponding CreateInputSource union member. [S15]

ServiceResource relates a service-local resourceIdentifier to the resource descriptor and optional input source. Resolve a global Resource identity only when sufficient account, Region, type and identifier evidence exists. Preserve unresolved or service-scoped identities otherwise. [S27]

ServiceFunction is a useful supporting entity because detailed findings refer to serviceFunctions. Preserve their source references and resolve them against the service function inventory. Do not invent direct resource-ARN references on Finding. [S17]

6. Assessments and findings

Entity	Fields and scope	Modeling rule
AssessmentEntity	assessmentId + serviceArn; assessmentStatus; assessmentStep; startedAt; endedAt; totalFindings; achievability; assessmentCost; billableAssessmentUnitCount; errorCode; errorMessage.	Service owns many runs. Keep optional values nullable. totalFindings is a run result, not the current open backlog.
FindingEntity	findingId scoped by serviceArn; name; description; reasoning; failureCategory; severity; status; policyComponent; comment; serviceFunctions; updatedAt; investigationDeepLink; three recommendation arrays.	Get/Update request scope supplies serviceArn when absent from the detail body. Current findings are service children, not mandatory children of an assessment.

AssessmentSummary requires assessmentId and serviceArn; the remaining listed properties are optional. The detailed Finding shape permits all properties to be absent. Promote identity from the request context when it is known, and treat truly unidentified responses as ingestion errors instead of silently manufacturing AWS identity. [S16] [S17]

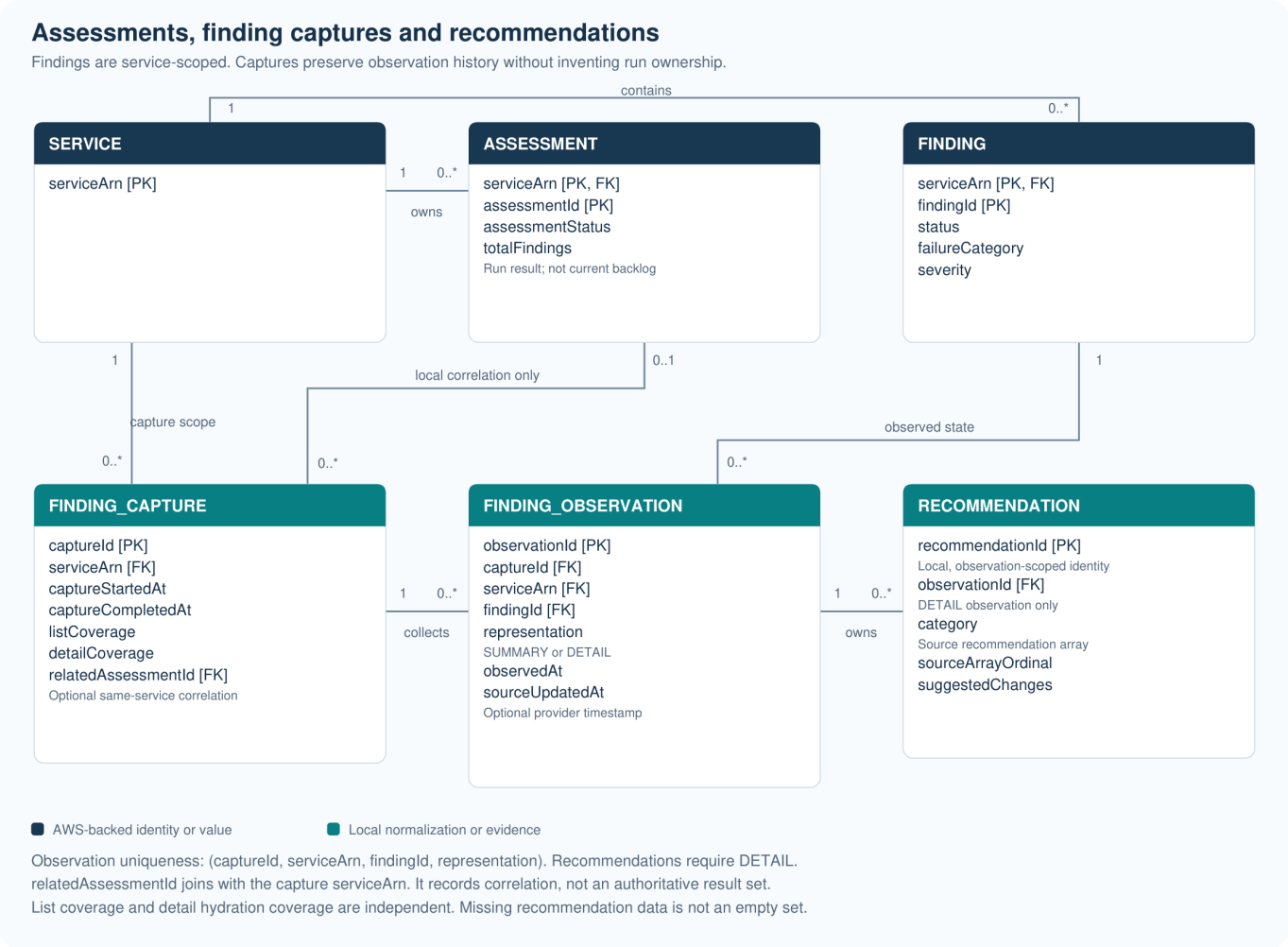
Dimension	Native values
Assessment execution	NOT_STARTED; PENDING; IN_PROGRESS; FAILED; SUCCESS
Finding disposition	OPEN; RESOLVED; IRRELEVANT
Finding severity	LOW; MEDIUM; HIGH
Failure category	SHARED_FATE; EXCESSIVE_LOAD; EXCESSIVE_LATENCY; MISCONFIGURATION_AND_BUGS; SINGLE_POINT_OF_FAILURE
Finding policy component	AVAILABILITY_SLO; MULTI_AZ_DISASTER_RECOVERY; MULTI_REGION_DISASTER_RECOVERY; DATA_RECOVERY

The assessment current-step field supports progress explanation but does not define a percentage-complete calculation. SUCCESS means successful execution. Assessment cost is the cost of the assessment, not a recommendation implementation estimate. [S16] [S18]

Do not create an authoritative Assessment-to-Finding foreign key. ListFailureModeFindings is filtered by service, severity, category and status; it has no assessmentId filter. GetFailureModeFinding also lacks that relationship. Historical finding bodies therefore require local captures, with any assessment association explicitly marked as correlation. [S19] [S20]

Finding disposition and evidence of remediation are different facts. AWS explains that the next assessment verifies a resolved fix, while an irrelevant finding stays suppressed across subsequent assessments. Keep both statuses visible in history and do not count suppression as validated repair. [S21]

7. Finding history and recommendations



Editable Mermaid source · SVG

Figure 3. A finding capture is a local collection interval. Its optional association with an assessment records when and why the collection occurred; it does not assert that AWS supplied an immutable finding set for that assessment.

RecommendationEntity as an owned value

The three native recommendation types are infrastructure/code, observability and testing. Each item contains only optional suggestedChanges[], with up to ten strings; each category array on a Finding permits up to five items. No recommendation ID, cost, complexity or standalone recommendation lifecycle is exposed. [S17] [S22]

Normalize each nested item as recommendationId, observationId, category, sourceArrayOrdinal, suggestedChanges[], payloadHash and capturedAt. Preserve grouping and order. A position or matching content hash does not establish a stable identity across future generated results.

If the UI needs title, implementation cost, effort, complexity, owner, work item or completion tracking, add RecommendationEnrichment or a separate RemediationTask. Retain estimate provenance,

units/currency, cost basis, confidence and estimation time. Several recommendation instances can link to one local remediation task without falsely merging provider entities.

FindingCapture should retain captureStartedAt, captureCompletedAt, filters, page coverage, collection outcome and optional relatedAssessmentId plus correlationBasis. Keep FindingObservation immutable within that capture, including updatedAt and raw source content. Do not infer first occurrence, resolution, or run ownership from a missing item or from a capture taken after a particular run.

Reviewed capture identity and detail coverage

Record	Reviewed key and integrity rule
`FindingCapture`	`captureId` is a local primary key; `serviceArn` scopes the collection. Record filters, list pagination outcome, detail hydration outcome and both collection intervals.
`FindingObservation`	`observationId` is a local primary key. `captureId` references its capture; `(serviceArn, findingId)` references the stable finding identity. The observation and capture must have the same `serviceArn`.
Observation uniqueness	One final immutable observation per `(captureId, serviceArn, findingId, representation)`, where representation is `SUMMARY` or `DETAIL`. Later fetches become new observations in a new capture, preserving the earlier evidence.
Assessment correlation	When present, `(FindingCapture.serviceArn, relatedAssessmentId)` references `(Assessment.serviceArn, assessmentId)`. Keep `correlationBasis`; the relationship never asserts exact finding ownership.
`Recommendation`	`recommendationId` is local. `observationId` references a `DETAIL` finding observation. Enforce uniqueness of `(observationId, category, sourceArrayOrdinal)` and preserve nested grouping.

`ListFailureModeFindings` and `GetFailureModeFinding` are separate reads. A fully paginated list does not mean all detail bodies have been fetched. Capture `sourceOperation`, `observedAt`, optional `sourceUpdatedAt`, field presence and the raw response for each representation. A failed detail hydration must not erase previously observed recommendations or become a known empty array. Recommendation completeness is known only for the detailed category fields actually returned. [S17] [S19] [S20]

Recommendation `category` identifies the source array: `INFRASTRUCTURE_AND_CODE`, `OBSERVABILITY`, or `TESTING`. It is distinct from `Finding.failureCategory`. Cost, complexity, estimated effort and display titles remain separately sourced application enrichment. Position and payload equality are useful change signals but do not prove that a future generated recommendation is the same provider entity.

8. Dependency configuration and discovery

DependencyDiscoveryConfig field	Type	Meaning
serviceArn	Local parent key	One optional current configuration value object per service.
status	Required native enum	INITIALIZING, ENABLED or DISABLED.
eligibleResourceCount	Nullable integer	Eligible compute-resource count; missing/unknown is distinct from zero.
message	Nullable string	Explanation returned by AWS; retain verbatim as diagnostic data.
updatedAt	Nullable timestamp	Provider update time; keep separate from collector observation time.

GetService and ListServices return the configuration object. CreateService and UpdateService accept only ENABLED or DISABLED as configuration intent; INITIALIZING is provider-managed. An observed state object must therefore not be reused as a write DTO. [S07] [S23]

Observed state	Operator interpretation
INITIALIZING + count unknown	Resource eligibility is still being established.
INITIALIZING + count 0	No eligible resources have been identified.
INITIALIZING + count greater than 0	Dependency analysis is in progress.
ENABLED + count 0	Enabled, with no eligible-resource coverage.
ENABLED + count greater than 0 + no message	Discovery results are ready for retrieval.
DISABLED	Discovery is turned off; old observations are historical evidence.

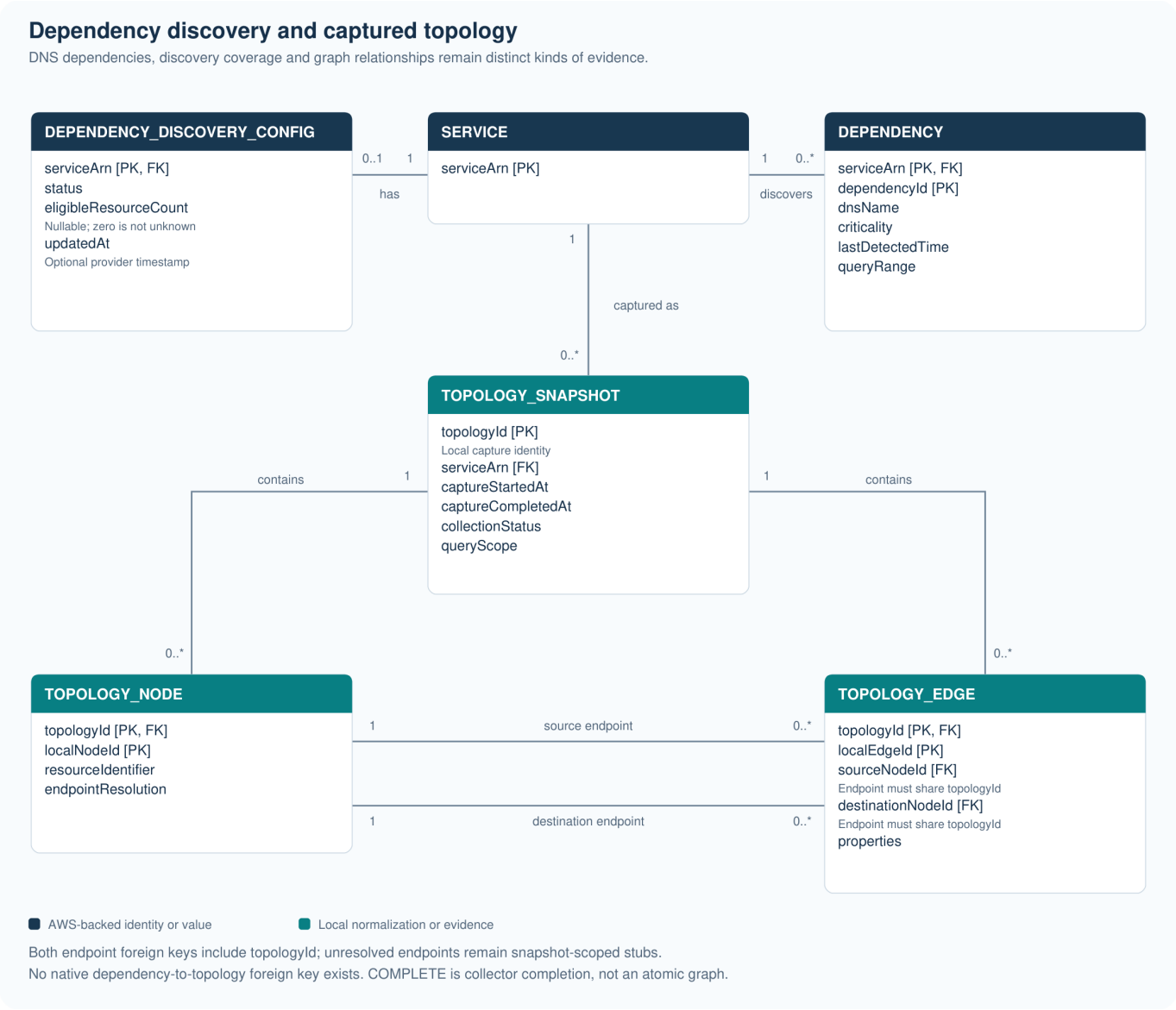
These interpretations follow the documented discovery-status combinations. Do not use a zero count or an empty list as proof that the architecture has no dependencies. [S24]

Resource discovery remains an independent process

Service.resourceDiscovery has its own status, lastRunAt, errorCode and errorMessage. Native statuses are RUNNING, SUCCEEDED, FAILED, COMPLETED_WITH_FAILURES and NOT_STARTED. Partial resource discovery should surface as partial coverage even when dependency discovery is enabled. [S25]

The UI should display discovery readiness, observed dependency inventory and evidence freshness independently. dependencyDiscovery.updatedAt is not a universal timestamp for the last DNS observation; individual dependencies have lastDetectedTime.

9. Dependencies and topology



Editable Mermaid source · SVG

Figure 4. Discovery and dependency records are AWS-backed. TopologySnapshot, TopologyNode and TopologyEdge are locally captured representations. There is no required relationship from a DNS dependency to a topology node or another managed service.

DependencyEntity uses (serviceArn, dependencyId), and retains dependencyName, dnsName, location, lastDetectedTime, sourceRegions[], criticality, provider?, comment? and queryRange. Criticality is HARD, SOFT or UNKNOWN. queryRange contains startTime, endTime, HOURLY/DAILY granularity and timestamped query counts. Store usage buckets with their query-window context. [S26]

UpdateDependency changes criticality and/or comment. Its response is smaller than DependencySummary; merge returned fields without discarding DNS or usage data. The current contract has no allowedStatus and no first-detection timestamp. If needed, model product-owned DependencyGovernance(allowedStatus, actor, reason, effective interval), and label firstObservedAt as the collector's first observation. [S28]

Do not equate DNS query counts with request success, latency or availability. Criticality represents the consequences of losing a dependency, not its current condition. Any relationship between a discovered

DNS name and a Resource or Service must have explicit matching evidence and a nullable resolution. [S29]

10. Topology snapshots and graph meaning

Object	Proposed fields and constraints
TopologyEntity / TopologySnapshot	local topologyId; serviceArn; captureStartedAt; captureCompletedAt; collectionStatus; queryScope; resourceDiscoveryLastRunAt?; resourceCount; unresolvedEndpointCount; edgeCount; contentHash?
TopologyNode	(topologyId, localNodeId); source resourceIdentifier; resource descriptor; optional resolved global resource key; resolution status and evidence
TopologyEdge	(topologyId, localEdgeId); sourceNodeId; destinationNodeId; raw endpoint identifiers/account/Region; properties[]
EdgeProperty	topologyType?; label?; preserve each returned property without treating a missing type as data flow

ListServiceTopologyEdges returns source and destination resource identifiers, optional account/Region values, and properties. It exposes no topology ID, capture timestamp, edge ID, resource count, assessmentId or atomic snapshot token. Generate local snapshot IDs and times, and state that COMPLETE means the collector finished its requested pages, not that AWS supplied a transactionally consistent graph. [S30]

Use ListResources for node inventory and retain unresolved edge endpoints as stubs. Record the filters used: ListResources defaults billable to true, which restricts its result to billable resources. Choose and verify the needed inventory scope deliberately. Count unique resolved inventory nodes separately from endpoint stubs and eligible compute resources. [S27]

Edge kind	Permitted UI interpretation
CONTAINMENT	Structural membership or hosting context; does not alone establish outage propagation.
DATA_FLOW	Directed data relationship; propagation still depends on criticality, redundancy and evidence.
OBSERVABILITY	Monitoring relationship; avoid treating the monitor as a required transaction dependency.
PERMISSIONS	Authorization relationship; not by itself a live call or failure path.

These are native topologyType values. AWS documents the optional label as present only for LLM-inferred edges. Preserve that provenance and avoid presenting inferred relationships as observed runtime traffic. [S31]

Historical accuracy: an edge collection and a resource collection can change while being paginated. Record both collection intervals and failure/coverage information. A later assessment does not retroactively turn the captured graph into that assessment's exact input graph.

Snapshot-scoped graph integrity

The node primary key is `(topologyId, localNodeId)` and the edge primary key is `(topologyId, localEdgeId)`. An edge must reference `(topologyId, sourceNodeId)` and `(topologyId, destinationNodeId)` in that **same** snapshot. Do not connect an edge to a current inventory node from another capture simply because its display name matches. Store endpoint raw account, Region and resource identifier evidence before resolution.

`resourceCount` counts unique inventory nodes within the recorded query scope; `unresolvedEndpointCount` counts separate stubs. Neither is the service's eligible-compute-resource count. `DependencyGovernance`, if introduced, and any DNS-to-resource matching are local overlays with their own provenance; they do not create an AWS foreign key or a live-health signal.

11. Events available from AWS

Next-generation Resilience Hub documents native EventBridge notifications with source aws.resiliencehub. Service ARNs appear in the envelope resources array; the envelope also carries event ID, time, account and Region. Delivery is best effort and can be delayed or missed. [S32]

Native detail type	Relevant detail	Refresh action
Failure Mode Assessment Completed	assessmentId; status SUCCESS; findingsCount; highSeverityCount	Refresh assessment list and service; capture current findings with qualified run correlation.
Failure Mode Assessment Failed	assessmentId; status FAILED; failureReason	Refresh run diagnostics; retain last completed posture with its age.
Failure Mode Assessment Queued	assessmentId	Track queued progress; poll the specific run through ListFailureModeAssessments.
Failure Mode Finding Resolved	findingId; status RESOLVED or IRRELEVANT; severity; category	Refresh the finding and relevant service aggregates.
New Dependency Discovered	dependencyId; dependencyName; dnsName; location; provider; sourceRegions	Refresh dependency inventory and coverage; invalidate affected graph matching.

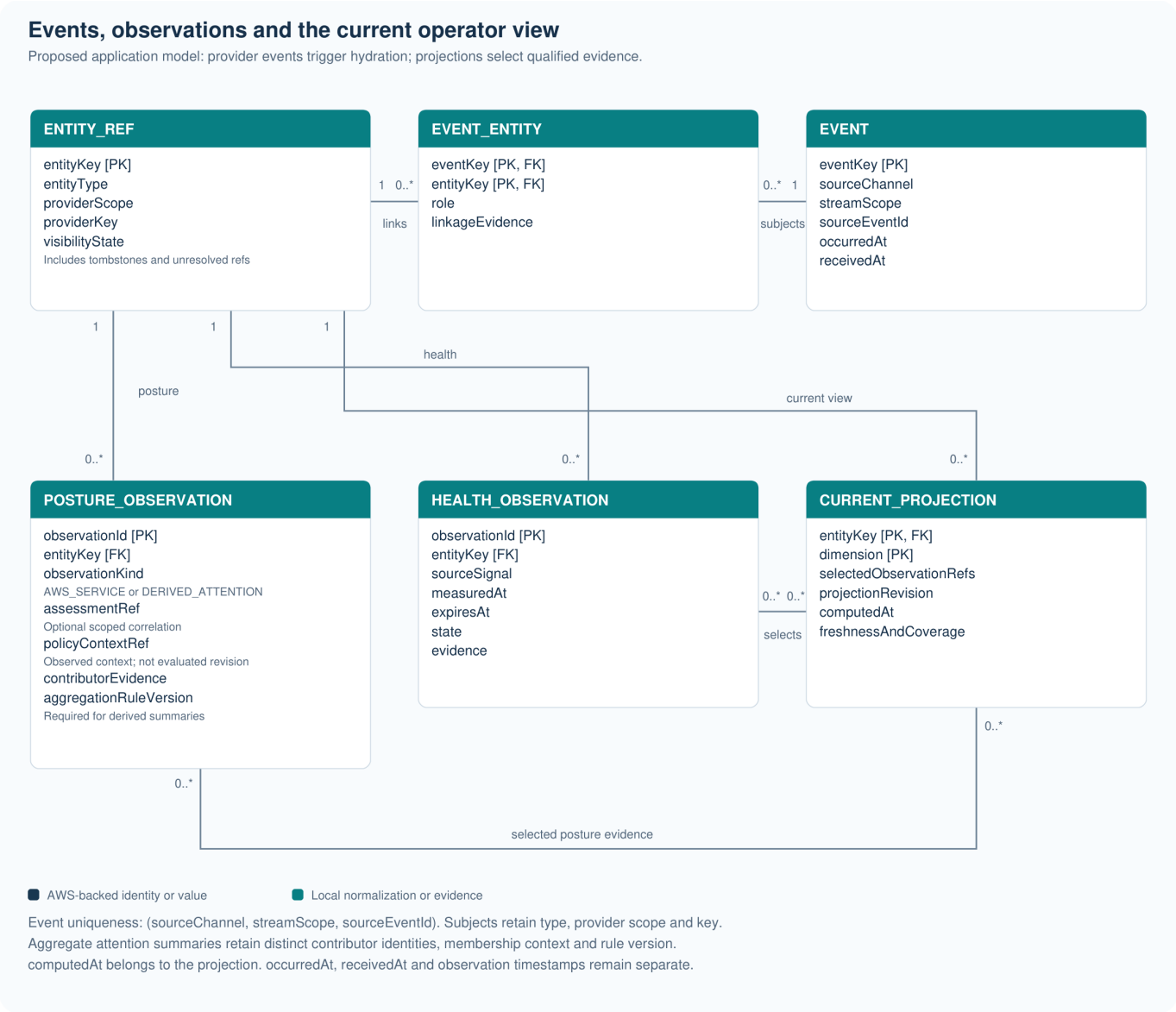
Assessment event shapes and counts are documented separately from AssessmentSummary. Preserve event-reported counts as event evidence, and reconcile them with hydrated API state. New-dependency events are emitted after a stabilization period, so event time is not necessarily first detection time. [S33] [S34] [S35] [S36] [S37]

Three event channels with different semantics

Native EventBridge notifications describe selected domain changes. They do not cover every finding transition or every configuration mutation. **ListServiceEvents, ListSystemEvents and ListPolicyEvents** expose entity timelines, with actor information and event details. **CloudTrail** records API calls, including console requests; a recorded request is not proof that an asynchronous workflow completed. [S38] [S39] [S40] [S41]

Keep these channels distinct in storage. The same logical change can appear in multiple channels with unrelated IDs. Treat timeline events as invalidations or evidence, not as a guaranteed complete stream of state transitions. Do not derive unsupported commands from event-type names: system-policy event labels, for example, do not create a policyArn field on System.

12. Temporal model for the operator view



Editable Mermaid source · SVG

Figure 5. Proposed local model for history, evidence and current UI state. EventEntity provides a many-to-many connection between events and typed subjects. Posture and health remain independently sourced.

Record	Essential fields
Event	eventKey; sourceChannel; streamOwner/scope; sourceEventId; eventType; occurredAt; receivedAt; actor?; raw payload; schema version
EventEntity	eventKey; entityKey; role; asserted/inferred relationship; linkage evidence
PostureObservation	entityKey; observationKind; native service achievability OR derived aggregate attention measures; scoped assessment correlation when known; observed policy-context reference; finding counts with scope; capture interval; completeness; provenance

Record	Essential fields
HealthObservation	subject; telemetry source and signal ID; observation window; measuredAt; receivedAt; expiresAt; health state; reason; evidence; aggregation-rule version
CurrentProjection	entityKey + dimension; typed selected evidence references; projectionRevision; freshness and coverage; computedAt

Uniqueness for provider events should include channel, stream namespace/owner and provider event ID. Keep occurredAt separate from receivedAt; arrival order is not event order. Deletions create tombstones so old observations still have resolvable subjects. Retain provenance for inferred links and preserve raw events for reprocessing.

Version membership and policy observations so a historical screen does not use today's topology or associations to explain yesterday's posture. Use observed validity intervals when that is all the evidence supports. Do not invent a provider effective time or evaluated policy version.

Typed subjects, observation kinds and projection times

`EntityRef` carries a local `entityKey`, `entityType`, verified `providerScope` and `providerKey`. A scope includes the enclosing service for findings, assessments and dependencies; it includes the enclosing system for journeys. Do not merge references based on display names or a hidden account assumption. Tombstones preserve historical references without suggesting that a deleted entity is still active.

A `PostureObservation` must distinguish `AWS_SERVICE` evidence from a `DERIVED_ATTENTION` summary. Native service evidence retains the returned component outcomes. A derived system or journey summary retains its distinct contributor service/finding identities, membership observation or capture reference, source observation references, coverage, and aggregation-rule version. It does not populate an AWS achievability field with an invented journey outcome.

`CurrentProjection.computedAt` is when the application assembled the view. It is separate from event `occurredAt` and `receivedAt`, source `observedAt`, provider update timestamps, and health measurement windows. Store typed selected evidence references so the UI can explain each badge. A late event may appear in the timeline without rolling back a newer hydrated value.

13. Status health posture and coverage

UI dimension	Data source	Display rule
Assessment progress	assessmentStatus and assessmentStep	Show queued/running/failed/completed separately from conclusions.
Resilience posture	Achievability per policy component	Native values: ACHIEVABLE or NOT_ACHIEVABLE. Missing results need an explicit unknown or unconfigured explanation.
Finding disposition	OPEN / RESOLVED / IRRELEVANT	Expose open risk, claimed remediation and suppression separately.
Resource discovery	resourceDiscovery status/errors	Show partial success and access failure as coverage conditions.
Dependency discovery	Config state, eligibility and message	Enabled is not sufficient evidence of useful coverage.
Operational health	External monitoring, SLO or incident evidence	Proposed local states: UNKNOWN / HEALTHY / DEGRADED / UNAVAILABLE, with evidence window and expiry.
Freshness and completeness	Collector timestamps, scope and success	Separate stale, partial and inaccessible from current healthy or achievable.

Achievability describes resilience targets based on assessment results. Resilience Hub's CloudWatch achievability metrics are emitted on assessment completion rather than continuously. Its dependency-discovery documentation distinguishes resilience validation from operational observability. Live health therefore needs an additional evidence source. [S42] [S43] [S44]

Rollup rules for systems and journeys

For the first UI version, aggregate **distinct services needing attention, open findings by severity, assessed-service coverage, and oldest evidence age**. Keep native component outcomes on each service. A journey-level resilience or availability conclusion requires a separately defined composition model and evidence; service membership alone supplies neither execution order nor redundancy.

Deduplicate portfolio counts by provider identity: one service can be in several journeys and systems. A failing strict effective objective is not proof of failure against every looser inherited requirement. A critical dependency indicates potential impact if unavailable; do not mark its consumer unavailable just because the dependency is classified HARD.

Example operator summary: Operational health: Healthy, measured 2 minutes ago. Resilience posture: Multi-Region recovery not achievable, assessed yesterday. Assessment: New run in progress. Discovery: Enabled, coverage partial. These facts can all be true at once; preserve the distinctions.

When the latest assessment fails, show that execution failure next to the previous successful posture and its age. When telemetry expires, health becomes unknown/stale according to the product rule; it must not remain green indefinitely.

Concrete aggregate rules for the first UI

1. **System service set:** take the union of service ARNs directly associated with the selected system, including services with no journey membership. Do not obtain the total by summing journey totals.

2. **Journey service set:** use the distinct service ARNs in that journey's constrained membership rows.

3. **Findings:** deduplicate by `(serviceArn, findingId)` before counting open severity. A shared service's finding can appear in each relevant journey view, but appears once in the enclosing system or portfolio total.

4. **Assessment coverage:** use distinct visible in-scope services as the denominator, and state how many have a successful assessment with sufficiently recent evidence. Distinguish not assessed, stale, inaccessible, and partial evidence; an unavailable denominator is not a zero denominator.

5. **Attention:** show the separate counts of services with unmet effective objectives, open findings, stale evidence, discovery gaps, or degraded telemetry. A combined attention total is the union of qualifying service identities, not the sum of those overlapping categories.

6. **Health:** compose system/journey health only under a named, versioned product rule with telemetry evidence and a validity window. Until a business composition model is defined, report impacted or unknown member services rather than a claimed end-to-end availability measurement.

A policy edit, association edit, or newer discovery observation can make a previous assessment's context less representative. Preserve the original observation and its age, record the current configuration separately, and expose the need for reassessment when supported by the returned state or an explicitly labelled local rule.

14. API read and mutation map

Domain	Read surface	Mutation surface
System	GetSystem; ListSystems	CreateSystem; UpdateSystem; DeleteSystem
User journey	GetUserJourney; ListUserJourneys	CreateUserJourney; UpdateUserJourney; DeleteUserJourney
Service	GetService; ListServices	CreateService; UpdateService; DeleteService
Membership	Service.associatedSystems and service list filters	CreateService / UpdateService.associatedSystems
Policy	GetPolicy; ListPolicies	CreatePolicy; UpdatePolicy; DeletePolicy
Input sources	ListInputSources	CreateInputSource; DeleteInputSource
Assessments	ListFailureModeAssessments	StartFailureModeAssessment
Findings	ListFailureModeFindings; GetFailureModeFinding	UpdateFailureModeFinding: status required; comment optional
Recommendations	Nested in detailed findings	No standalone recommendation mutation in the current contract
Discovery config	GetService; ListServices	CreateService / UpdateService.dependencyDiscovery
Dependencies	ListDependencies	UpdateDependency: criticality and/or comment
Resources / topology	ListResources; ListServiceTopologyEdges	No topology-object CRUD in the examined operation inventory
Entity timelines	ListSystemEvents; ListServiceEvents; ListPolicyEvents	Provider-generated; local notes remain separate

This map uses the generated v2 SDK operation inventory and operation schemas. It intentionally does not require GetFailureModeAssessment, StartServiceTopologyDiscovery, ClassifyDependency or CreateResiliencePolicy, which are absent from that inventory despite appearing in some guide examples. [S02] [S07] [S09] [S19] [S20] [S28] [S45] [S46]

Command handling

Represent a user operation as a local command with a command ID, target identity, input, actor, submitted time, request result and reconciliation state. Reuse clientToken for retries only on operations that expose it, such as starting an assessment. After success, merge the returned projection and re-read the affected entities as needed. Do not optimistically mark an assessment complete or claim a finding fix has been independently verified. [S45]

Separate confirmed provider values from pending edits. For association arrays, policy detachment, clearing optional fields and concurrent edits, validate exact semantics with the chosen SDK and controlled integration calls before exposing destructive editing behavior.

15. Event processing and implementation checks

Recommended ingestion loop

1. Bootstrap. Enumerate visible systems, services and policies; load journeys and required service children. Complete pagination before replacing a collection. Record filters, account/Region and visibility scope.

2. Receive. Persist the raw event, deduplicate within its source stream, and record occurrence and receipt times. Resolve subject references without discarding unresolved ones.

3. Hydrate. Use the event as a targeted refresh trigger. Read the authoritative service, run, finding or dependency state. Use the recorded assessment ID when matching ListFailureModeAssessments results.

4. Capture. Append new observations with collection intervals and provenance. Correlate finding or topology captures to a run only as explicitly qualified local evidence.

5. Project. Recompute affected service and aggregate views from current evidence; advance a local projection revision and push that revision to connected UI clients.

6. Reconcile. Poll active workflows and periodically refresh inventories and entity timelines independently of EventBridge. Retain last trustworthy data and expose a stale/partial indicator when refresh fails. Best-effort delivery makes this reconciliation necessary. [S32]

Acceptance checks for the model and UI

Scenario	Expected behavior
A service joins two systems and several journeys	One service identity; constrained membership joins; no duplicate portfolio findings.
An assessment completes with high-severity findings	Execution shows success; posture and findings can still require attention.
An operator marks a finding irrelevant	Suppression remains distinct from a verified remediation and is preserved in history.
Discovery is enabled with zero eligible resources	Show coverage gap, not a successful dependency-free architecture.
A page is missing or a permission check fails	Do not remove unseen objects or reset counts to zero; flag incomplete collection.
An old event arrives after a newer observation	Retain event history; avoid rolling back the current projection from stale payloads.
A policy or membership changes during a run	Keep old observations and current configuration separate; qualify evaluation-context correlation.
An edge endpoint cannot be resolved	Keep an endpoint stub and evidence, without assigning an invented resource or service.
A summary omits detail-only properties	Preserve previously hydrated fields and their timestamps.

16. Official sources

All 48 sources below are AWS primary documentation. Reviewed on **5 November 2026**. The generated v2 operation and shape contracts take precedence over inconsistent overview terminology. Sources document the API; the local identities, temporal records and composition rules are the design proposed in this report.

ID	Official source
S01	User guide conceptual data types
S02	Official SDK operation and shape inventory for resiliencehubv2
S03	System
S04	SystemSummary
S05	AssociatedSystem and nested journey membership
S06	Service and GetService response
S07	CreateService and required configuration
S08	GetPolicy
S09	UpdateService association and discovery input
S10	MultiRegionTargets
S11	Policy inheritance and multi-level evaluation
S12	EffectivePolicyValues
S13	SloSource ownership provenance
S14	PermissionModel
S15	ListInputSources
S16	AssessmentSummary
S17	Finding
S18	AssessmentCost
S19	ListFailureModeFindings
S20	GetFailureModeFinding
S21	Viewing findings and managing disposition
S22	InfrastructureAndCodeRecommendation
S23	DependencyDiscoveryConfig

ID	Official source
S24	Monitoring discovery status
S25	ResourceDiscoveryStatus
S26	DependencySummary
S27	ListResources and billable default
S28	UpdateDependency
S29	How dependency discovery works
S30	ListServiceTopologyEdges
S31	EdgePropertySummary
S32	Native EventBridge notification support and delivery
S33	Assessment completed event
S34	Assessment failed event
S35	Assessment queued event
S36	Finding resolved or irrelevant event
S37	Dependency discovered event and stabilization
S38	ListServiceEvents
S39	ListSystemEvents
S40	ListPolicyEvents
S41	CloudTrail logging
S42	Achievability
S43	CloudWatch achievability metrics emission
S44	Dependency discovery and operational observability
S45	StartFailureModeAssessment
S46	UpdateFailureModeFinding
S47	UserJourney
S48	ListFailureModeAssessments

Implementation decisions to verify

Validate association-array replacement, policy and component detachment, account-specific visibility, pagination behavior, evidence freshness rules, resource-to-dependency matching and the business rules for composing journey health. The broader v2 API also exposes tests, test runs, assertions and reports; these can be added as evidence entities when the UI includes resilience validation workflows. [S02]

Scope and evidence limits

This is a documentation-based design analysis, not a validation of a particular AWS account. No account-specific API responses or permissions were supplied. Proposed local keys, captures, governance fields, health states, rollups and ingestion rules are distinguished from provider fields throughout. The model should be verified against the SDK version and controlled payloads used during implementation. The accompanying application uses fictional operational data; it does not connect to AWS or establish actual runtime health.

Diagram maintenance

The `.mmd` files are the editable semantic sources. `diagrams/diagram-layouts.json` supplies SVG presentation coordinates and labels. Run `python scripts/render_diagrams.py` after changing a diagram, then `python scripts/build_report.py` to rebuild the PDF from this Markdown source. No remote diagram renderer or CDN is used.